



Московский государственный университет имени М.В. Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра автоматизации систем вычислительных комплексов

Кибизов Кирилл Олегович

**Разработка методов определения задержки при
нагрузочном тестировании при передаче данных между
графическими процессорами в вычислительном кластере**

КУРСОВАЯ РАБОТА

Научный руководитель:

к.ф.-м.н., доцент

Сальников Алексей Николаевич

Москва, 2026

Аннотация

Разработка методов определения задержки при нагрузочном тестировании при передаче данных между графическими процессорами в вычислительном кластере

Кибизов Кирилл Олегович

В настоящей курсовой работе рассмотрена задача измерения задержки передачи данных между графическими процессорами (GPU) в вычислительном кластере. Измерения выполнены для внутриузловых и межузловых пар GPU при варьировании трёх параметров эксперимента: размера сообщения, среды передачи и режима тестирования. Среда передачи определяет, выполняется ли обмен напрямую между GPU или через оперативную память хоста. Режим тестирования задаёт способ организации обменов: изолированную передачу между одной парой GPU или одновременные передачи между всеми парами GPU.

В рамках работы реализован программный модуль, позволяющий измерять задержки передачи данных между GPU и представлять результаты в форме, удобной для визуального анализа неоднородностей коммуникационной среды кластера. По результатам измерений построены графики зависимости задержки от размера сообщения, а также теплокарты, показывающие распределение задержек по парам GPU.

Содержание

1	Введение	4
1.1	Актуальность	4
1.2	Модель коммуникационной среды	4
1.3	Режимы тестирования	8
2	Постановка задачи	10
2.1	Проблема	10
2.2	Цель работы	10
2.3	Задача	10
3	Обзор	12
3.1	Существующие инструменты	12
4	Методы измерения	15
4.1	Способы фиксации интервала задержки	15
4.2	Таймеры	16
5	Программная реализация	19
5.1	Общая архитектура модуля	19
5.2	Учёт среды передачи	20
5.3	Подготовка MPI-окружения	21
5.4	Реализация режима one_to_one	22
5.5	Реализация режима all_to_all	24
6	Методика эксперимента	27
6.1	Этапы	27
6.2	Выбор количества итераций	27
6.3	Обработка результатов	27
6.4	Ограничения	28

7	Среда проведения эксперимента	29
7.1	Аппаратная среда	29
7.2	Программная среда	30
8	Результаты эксперимента	31
8.1	Зависимость от размера сообщения	31
8.2	Зависимость от режима тестирования	33
	Заключение	34
	Литература	36
	Приложение А – Запуск программного модуля	39

1 Введение

1.1 Актуальность

Современные вычислительные кластеры применяются в задачах, требующих значительных вычислительных ресурсов, например, для обучения моделей машинного обучения и для численного моделирования. Для ускорения таких вычислений в узлах кластера применяются графические процессоры (GPU), рассчитанные на массовое параллельное выполнение однотипных операций над данными.

При использовании нескольких GPU производительность зависит не только от скорости вычислений, но и от скорости передачи данных между GPU. Так, медленный обмен данными может стать узким местом распределённых вычислений.

Дополнительная сложность состоит в том, что коммуникационная среда вычислительного кластера, как правило, неоднородна: передачи между отдельными парами GPU могут задействовать разные участки внутриузловой и межузловой сети, и соответствующие задержки могут заметно различаться. Для анализа такой неоднородности необходим инструмент, позволяющий измерять задержки и представлять результаты в виде, удобном для их дальнейшей интерпретации.

Для дальнейшего описания таких передач введём модель коммуникационной среды.

1.2 Модель коммуникационной среды

Графовое представление коммуникационной среды

Для формального описания передач данных между GPU коммуникационная среда рассматривается как совокупность взаимодействующих компонент. В рамках настоящей работы такая система представляется в виде графа

$$G = (V, E),$$

Вершины графа G соответствуют компонентам:

- внутри вычислительного узла — хостам H (CPU + RAM), устройствам D (GPU + VRAM), сетевым картам NIC, PCIe-коммутаторам и коммутаторам NVSwitch;
- в межузловой сети — коммутаторам Ethernet и InfiniBand.

Рёбра графа E соответствуют физическим соединениям.

- К внутриузловым относятся PCIe, NVLink и межпроцессорные соединения.
- К межузловым — Ethernet и InfiniBand.

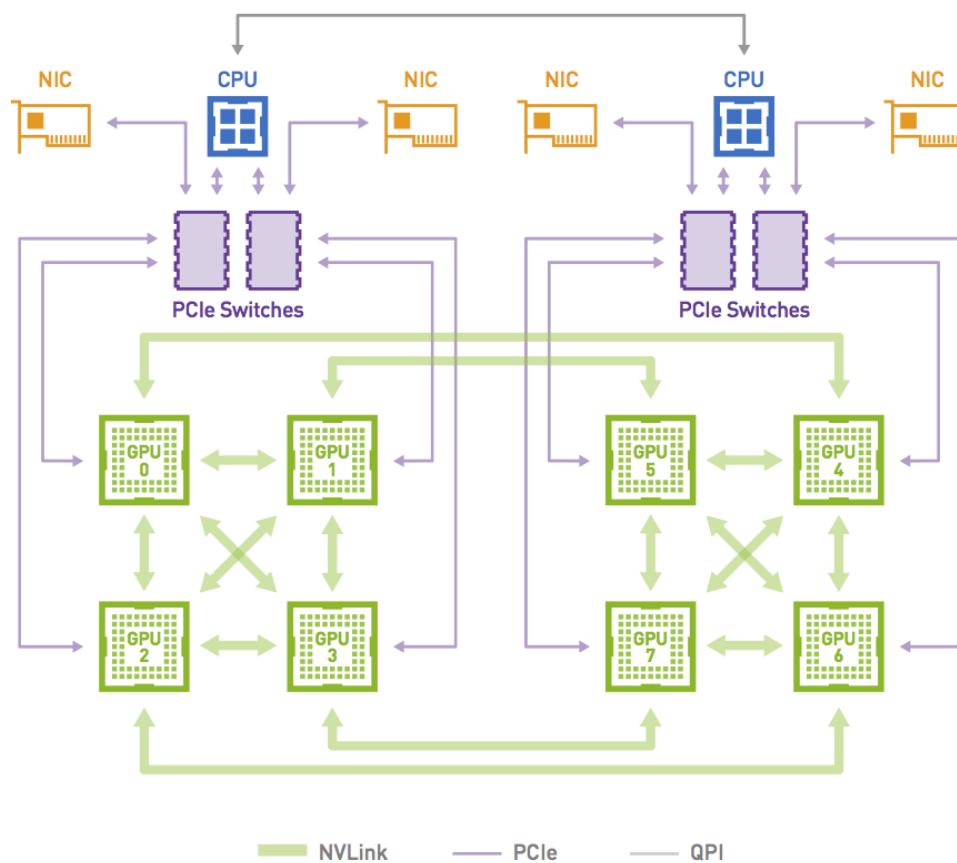


Рис. 1: Пример коммуникационной среды (Источник: [12])

В данной работе рассматриваются только те кластеры, для которых эта модель коммуникационной среды применима.

Сообщение

Под *сообщением* будем понимать буфер синтетически сформирован-

ных данных размера m , передаваемый от устройства-источника D_{src} к устройству-приёмнику D_{dst} .

Маршрут передачи

Пусть *маршрутом* в графе G называется последовательность вершин и рёбер, по которой может идти передача данных от D_{src} к D_{dst} . Для каждой пары устройств физическая топология задаёт множество возможных маршрутов. Фактический маршрут передачи определяется топологией системы, исполняемым кодом и коммуникационным стеком.

Важно отметить, что в общем случае сообщение не передаётся по сети как единый объект: на каждом участке маршрута оно может быть разбито на фрагменты согласно правилам используемого протокола передачи. Таким образом, из-за фрагментации, передача сообщения описывается, вообще говоря, не одним маршрутом, а некоторым подмножеством множества допустимых маршрутов.

Внутриузловые маршруты

Под *прямым обменом* между GPU далее понимается передача без промежуточного копирования в оперативную память хоста. В рассматриваемой системе с GPU NVIDIA такой обмен может выполняться средствами GPUDirect Peer-to-Peer (GPUDirect P2P).

Ниже в формулах подпись над стрелкой указывает тип маршрута. Обозначение PCIe соответствует передаче через шину PCI Express, включая линии PCIe и PCIe-коммутаторы. Обозначение NVLink/NVSwitch соответствует передаче через межсоединение GPU на базе NVLink, в том числе с использованием коммутаторов NVSwitch.

При прямом обмене через PCIe маршрут можно записать в виде:

$$D_{src} \xrightarrow{\text{PCIe}} D_{dst}$$

Если между устройствами доступен маршрут через NVLink или NVSwitch, он, как правило, предпочтительнее, поскольку обеспечивает боль-

шую пропускную способность по сравнению с PCIe:

$$D_{src} \xrightarrow{\text{NVLink/NVSwitch}} D_{dst}$$

Если прямой обмен между GPU недоступен или отключён, передача выполняется посредством оперативной памяти хоста:

$$D_{src} \xrightarrow{\text{PCIe}} H \xrightarrow{\text{PCIe}} D_{dst}$$

В многопроцессорных узлах маршрут через память хоста зависит от NUMA-топологии. Иначе говоря, GPU, NIC и область памяти хоста могут быть привязаны к разным CPU. В таком случае передача может дополнительно задействовать межпроцессорное соединение.

Межузловые маршруты

Под net далее понимается межузловая сеть: коммутаторы Ethernet или InfiniBand и соединяющие их каналы связи.

При поддержке GPUDirect RDMA (Remote Direct Memory Access) сетевая карта может напрямую обращаться к памяти GPU, поэтому промежуточное копирование через оперативную память хоста не требуется:

$$D_{src} \xrightarrow{\text{PCIe}} \text{NIC}_{src} \xrightarrow{\text{net}} \text{NIC}_{dst} \xrightarrow{\text{PCIe}} D_{dst}$$

Если используется промежуточное копирование через память хоста, маршрут имеет вид:

$$D_{src} \xrightarrow{\text{PCIe}} H_{src} \xrightarrow{\text{PCIe}} \text{NIC}_{src} \xrightarrow{\text{net}} \text{NIC}_{dst} \xrightarrow{\text{PCIe}} H_{dst} \xrightarrow{\text{PCIe}} D_{dst}$$

Среды передачи

Среда передачи в настоящей работе формализует ограничение на множество допустимых маршрутов в G . Рассматриваются две среды:

- auto — маршрут без промежуточного копирования в память хоста, при условии что он поддерживается. В ином случае транспортная библиотека автоматически использует маршрут через оперативную

память хоста.

- `host` — маршрут обязательно содержит оперативную память хоста.

Задержка передачи

Сквозной задержкой передачи для упорядоченной пары устройств (D_{src}, D_{dst}) будем называть временной интервал, соответствующий полной передаче сообщения от D_{src} к D_{dst} .

На практике такой интервал не всегда измеряется напрямую. Поэтому далее под *измеренной задержкой передачи* понимается оценка сквозной задержки, полученная выбранным способом фиксации временного интервала. Конкретные способы выбора начала и конца измеряемого интервала рассматриваются в подразделе 4.1.

Причины повышенных задержек

Под *причинами повышенных задержек* будем понимать те факторы, из-за которых измеренная задержка для некоторой пары GPU оказывается больше, чем для других пар при сопоставимых условиях. К основным факторам относятся:

- Топология и конфигурация маршрута: сложность маршрута, в том числе из-за необходимости промежуточного копирования через оперативную память хоста;
- Конкуренция за общие ресурсы: насыщение каналов связи и рост очередей при одновременных передачах;
- Деградация оборудования и отказы: нестабильность отдельных линий связи, аппаратные неисправности, ошибки драйверов и системного ПО.

1.3 Режимы тестирования

Режим тестирования задаёт сценарий обмена между GPU: какие упорядоченные пары устройств участвуют в обмене, и происходят ли передачи одновременно или изолированно.

- `one_to_one` — в каждый момент времени активна только одна пара GPU, остальные GPU простаивают.
- `all_to_all` — одновременно активны передачи между всеми рассматриваемыми парами GPU, что порождает конкуренцию за общие ресурсы.

2 Постановка задачи

2.1 Проблема

Задержки передачи для внутриузловых и межузловых пар GPU неоднородны. Они зависят от размера сообщения, среды передачи и режима тестирования. Для их сравнения требуется измерять задержки для всех GPU в сопоставимых условиях.

2.2 Цель работы

Целью работы является исследование задержек передачи данных для внутриузловых и межузловых пар GPU в зависимости от размера сообщения, среды передачи и режима тестирования. Для проведения такого исследования разрабатывается программный модуль, обеспечивающий измерение задержек для всех пар GPU в сопоставимых условиях.

2.3 Задача

Пусть $\mathcal{D} \subset V$ — множество GPU исследуемого кластера в модели $G = (V, E)$ из подраздела 1.2. Зафиксируем параметры эксперимента:

- множество размеров сообщений $M = \{m_1, m_2, \dots, m_k\}$;
- множество сред передачи $ENV = \{\text{auto}, \text{host}\}$;
- множество режимов тестирования $MODE = \{\text{one_to_one}, \text{all_to_all}\}$.

Для каждой упорядоченной пары устройств (D_{src}, D_{dst}) , каждого размера сообщения m , каждой среды передачи env и каждого режима тестирования $mode$ выполняется серия измерений задержки передачи данных. Результатом такой серии является временной ряд

$$\{t_i\}_{i=1}^n = t_1, t_2, \dots, t_n,$$

где t_i — задержка передачи на i -й итерации измерения, а n — общее число итераций.

Значения t_i зависят как от выбранных параметров эксперимента, так и от неконтролируемых факторов, например от фоновой нагрузки на кластер и состояний внутренних очередей коммутаторов.

По полученному временному ряду вычисляются статистические характеристики задержки: минимум, максимум, среднее, медиана, выборочная дисперсия и выборочное стандартное отклонение.

3 Обзор

3.1 Существующие инструменты

В данном подразделе рассматриваются существующие инструменты для измерения задержек и пропускной способности при обменах между GPU. Сравнение проводится по следующим критериям:

- область применения: внутриузловые, межузловые передачи;
- возможность выбора среды передачи (auto или host) как независимого параметра эксперимента;
- поддержка нагрузочных режимов тестирования, в первую очередь one_to_one и all_to_all;
- форма результата: агрегированные характеристики, временные ряды, попарные матрицы задержек.

`p2pBandwidthLatencyTest`

Утилита `p2pBandwidthLatencyTest` [16], входящая в состав `CUDA Samples`, измеряет задержку и пропускную способность при передаче данных между парами GPU внутри одного вычислительного узла и представляет результаты в виде попарных матриц.

В контексте настоящей работы утилита крайне ограничена. Во-первых, она работает только с внутриузловыми передачами и не охватывает межузловую сеть. Во-вторых, отсутствует режим одновременной нагрузки на несколько пар.

`nvbandwidth`

Инструмент `nvbandwidth` [17], разрабатываемый NVIDIA, предназначен для измерения пропускной способности и задержки при передаче данных между CPU и GPU, а также между парами GPU.

Однако у `nvbandwidth` есть два важных ограничения. Во-первых, межузловой режим в актуальной реализации ориентирован на системы с Multi-Node NVLink, где GPU из разных узлов могут напрямую обмениваться дан-

ными через сеть NVLink. Поэтому этот режим не подходит как универсальный инструмент для измерения межузловых передач через InfiniBand или Ethernet. Во-вторых, в `nvbandwidth` нет режима, который одновременно нагружал бы все пары GPU и при этом строил попарную матрицу задержек: при одновременной нагрузке инструмент выводит агрегированные характеристики, а не задержки для каждой пары устройств.

OSU Micro-Benchmarks

OSU Micro-Benchmarks (OMB) [18], разрабатываемый в Ohio State University, — набор тестов для измерения задержки и пропускной способности операций MPI. При сборке с поддержкой CUDA этот инструмент позволяет выбрать расположение коммуникационных буферов: в памяти хоста или в памяти устройства.

Но у OMB есть два ограничения. Во-первых, OMB не ориентирован на построение полной попарной матрицы задержек между GPU. Обычный тест задержки измеряет обмен между двумя выбранными MPI-процессами и возвращает характеристику только для этой пары. Многопроцессные тесты OMB позволяют создать одновременную нагрузку, однако их результат обычно представлен в агрегированном виде и не содержит отдельных значений задержек. Во-вторых, OMB не позволяет явно зафиксировать среду передачи как отдельный параметр эксперимента, например для сравнения обмена через память хоста. Поэтому в OMB нельзя в явном виде сравнить для одной и той же пары GPU два режима маршрутизации, аналогичные `auto` и `host` в настоящей работе.

Модуль в составе Clustbench

Проект Clustbench [19] предназначен для тестирования и анализа коммуникаций в вычислительных кластерах. Наиболее близким к рассматриваемой задаче является входящий в него модуль, описанный в работе [1]. Этот модуль предназначен для измерения характеристик передачи данных между GPU и поддерживает два режима тестирования: `one_to_one` и `all_to_all`. Оба режима построены по схеме «один MPI-процесс на один узел», при которой один процесс управляет всеми GPU, доступными ему на данном узле в рамках запуска.

В модуле учитываются как внутриузловые, так и межузловые пары GPU. Для каждой пары и каждого размера сообщения вычисляются статистические характеристики задержки. Результаты сохраняются в формате NetCDF и могут быть представлены в виде матриц: строки соответствуют GPU-источникам, столбцы — GPU-приёмникам, а элемент матрицы содержит значение выбранной статистики задержки для соответствующей пары устройств.

Главное ограничение для постановки настоящей работы состоит в том, что среда передачи не выделена как независимый параметр эксперимента.

Настоящая работа развивает подход Clustbench. В частности, вводится явный выбор среды передачи (auto/host), и используется иная схема «один MPI-процесс на один GPU», при которой каждый процесс закреплён за своим GPU.

4 Методы измерения

4.1 Способы фиксации интервала задержки

После введения понятия сквозной задержки (см. подраздел 1.2) возникает вопрос о том, как фиксировать начало и конец соответствующего временного интервала на практике.

Измерение по отметкам отправителя и получателя

В строгой постановке задержка для упорядоченной пары устройств (D_{src}, D_{dst}) соответствует интервалу от начала передачи сообщения на стороне отправителя до завершения его приёма на стороне получателя. Иначе говоря, отправитель фиксирует момент начала передачи, получатель — момент завершения приёма, после чего задержка вычисляется как разность этих отметок. Такой подход кажется наиболее естественным и очевидным, однако на практике начальная и конечная отметки могут фиксироваться разными процессами, расположенными на разных узлах вычислительного кластера. Поэтому нетривиальной проблемой становится синхронизация времени между узлами: даже небольшое расхождение локальных часов может существенно исказить результат измерения.

Оценка по схеме ping-pong

В случае схемы ping-pong сообщение сначала передаётся от первого участника ко второму, а затем возвращается. Фиксируется суммарное время обмена в двух направлениях, а оценка односторонней задержки получается делением этой величины на два. Такой подход предполагает, что условия передачи в прямом и обратном направлениях сопоставимы. Если же маршруты или условия передачи различаются, полученная оценка задержки оказывается смещённой.

Измерение с подтверждением приёма

Также возможна схема с подтверждением приёма. В этом случае отправитель фиксирует момент начала передачи и ожидает короткое подтверждение ask от получателя, означающее завершение приёма. Такой подход поз-

воляет проводить измерение целиком на стороне отправителя, без необходимости синхронизации часов. Однако итоговое значение включает не только передачу основного сообщения, но и время доставки подтверждения `ack`.

Измерение на стороне отправителя

При измерении на стороне отправителя фиксируется время операций, выполняемых для отправки данных. Однако завершение этих операций не означает, что данные уже доступны приёмнику. Например, завершение `MPI_Send()` или ожидание `MPI_Isend()` через `MPI_Wait()` гарантирует возможность повторного использования буфера отправителя, но не завершение приёма данных. Поэтому на стороне отправителя в общем случае нет события, однозначно соответствующего моменту готовности данных у получателя.

Измерение на стороне получателя

При измерении на стороне получателя фиксируется интервал от готовности принять данные до завершения операции приёма. Завершение приёма имеет однозначный смысл: к этому моменту данные уже записаны в указанный буфер и готовы к использованию. Для `MPI_Recv()` таким событием является возврат из функции, а для схемы `MPI_Irecv()` с `MPI_Wait()` — успешный возврат из `MPI_Wait()`. Однако при этом измеренный интервал может включать не только саму передачу, но и ожидание её начала, например время подготовки данных на стороне отправителя.

В настоящей работе используется именно этот подход.

4.2 Таймеры

CUDA events

Механизм `cudaEventRecord()` / `cudaEventElapsedTime()` позволяет измерять интервалы между операциями, выполняемыми в CUDA-потоках. Событие, поставленное вызовом `cudaEventRecord()`, фиксируется не в момент вызова на CPU, а после того, как GPU дойдёт до соответствующей позиции в указанном потоке. Поэтому CUDA events удобны для измерения операций, явно поставленных в этот поток.

В рассматриваемой задаче применение CUDA events затруднено по двум причинам. Во-первых, при использовании CUDA-aware MPI транспортная библиотека может использовать внутренние CUDA-потоки для GPU-операций, к которым код не имеет доступа, поэтому поставить события вокруг самой передачи не представляется возможным. Во-вторых, при передаче через GPUDirect RDMA данные перемещаются из памяти GPU непосредственно через NIC аппаратным DMA-движком, минуя CUDA-потоки — в таком случае CUDA events в принципе не могут зафиксировать момент завершения передачи. В результате с помощью CUDA events можно измерить лишь явные cudaMemcpy-вызовы, но не интервал передачи между MPI-процессами в целом.

Системные таймеры операционной системы

Стандартным средством измерения времени на стороне CPU является функция `clock_gettime()` с источником `CLOCK_MONOTONIC`. Этот источник предоставляет монотонную временную шкалу: его отсчёты не убывают и не подвержены резким изменениям системного времени, выполняемым службами синхронизации. Однако сам ход `CLOCK_MONOTONIC` может немного корректироваться системой, то есть замедляться или ускоряться. Разрешение современных монотонных таймеров обычно составляет от единиц до десятков наносекунд, что достаточно для задержек, рассматриваемых в настоящей работе.

Таймер MPI

Функция `MPI_Wtime()` возвращает значение таймера, доступного каждому MPI-процессу, и предназначена для измерения прошедшего астрономического времени внутри MPI-программы. В реализации Open MPI на POSIX-платформах в качестве источника времени может использоваться системный монотонный таймер, например `clock_gettime()`, поэтому по смыслу `MPI_Wtime()` близка к системным CPU-таймерам. Разрешение `MPI_Wtime()` определяется реализацией MPI и может быть получено с помощью функции `MPI_Wtick()`. При этом стандарт MPI не требует глобальной согласованности таймера между процессами, вследствие чего отметки `MPI_Wtime()`, полученные разными процессами, в общем случае нельзя напрямую сравнивать.

При «измерении на стороне получателя» (см. подраздел 4.1) глобальная согласованность системных таймеров не требуется. В программной реализации используется `MPI_Wtime()`.

5 Программная реализация

5.1 Общая архитектура модуля

Программный модуль реализован на C++ с использованием CUDA и MPI.

CUDA — программная платформа NVIDIA для разработки программ, выполняемых на графических процессорах. В модуле используется CUDA Runtime API [9], через который выполняются выбор устройства, выделение и освобождение его памяти, копирование данных и синхронизация операций.

MPI — стандарт обмена сообщениями между параллельными процессами, запущенными как на одном вычислительном узле, так и на разных узлах кластера.

В работе используется Open MPI [7] — реализация стандарта MPI с включённой поддержкой CUDA-aware режима. В этом режиме CUDA-буферы могут передаваться непосредственно в MPI-операции без явного промежуточного копирования в память хоста.

В качестве транспортного слоя используется UCX [8]. Эта библиотека выбирает способ передачи данных в зависимости от своей конфигурации, топологии системы и характеристик передаваемого сообщения.

Основные файлы программного модуля, отвечающие за измерения, обработку и визуализацию результатов, перечислены ниже:

- `gpu_benchmark.cpp` — инициализация MPI и CUDA, сбор топологии;
- `gpu_common.cpp` — разбор аргументов командной строки, запись сырых временных рядов;
- `gpu_one_to_one.cpp` — логика режима изолированного попарного обмена;
- `gpu_all_to_all.cpp` — логика режима одновременного обмена всех со всеми;
- `gpu_heatmap.py` — построение теплокарт задержек;
- `gpu_plot.py` — построение графиков зависимости задержек от размера для отдельных пар GPU;

- `gpu_netcdf.py` — экспорт результатов в формат NetCDF [15].

5.2 Учёт среды передачи

Пользовательский параметр `--env` задаёт среду передачи данных. От выбранного значения зависят параметры UCX, устанавливаемые перед инициализацией MPI.

	auto	host
UCX_TLS	cuda_copy, cuda_ipc, <IB>, cma, sm, self	<IB>, cma, sm, self
UCX_IB_GPU_DIRECT_RDMA	y	n

Таблица 1: Различия параметров UCX в режимах auto и host.

- `cuda_copy` — работа с памятью GPU средствами CUDA Runtime API;
- `cuda_ipc` — прямой обмен между GPU на одном узле через CUDA Inter-Process Communication (посредством PCIe или NVLink);
- `IB(rc_verbs, rc_mlx5, ud_verbs, ud_mlx5, dc_mlx5)` — транспорты InfiniBand с поддержкой RDMA;
- `cma` — внутриузловое копирование между адресными пространствами процессов;
- `sm` — внутриузловая передача через разделяемую память хоста;
- `self` — локальная передача данных внутри одного процесса.

Стоит отметить, что в таблице указывается не порядок выбора транспортов, а множество транспортов, разрешённых для данной среды передачи. Фактический транспорт выбирается UCX во время выполнения с учётом типа памяти, расположения процессов и параметров передаваемого сообщения.

Флаг `UCX_IB_GPU_DIRECT_RDMA=y` разрешает сетевой карте читать и писать GPU-память напрямую через RDMA, минуя промежуточный буфер в хосте.

В режиме `host` из `UCX_TLS` исключаются CUDA-транспорты, поэтому UCX приходится работать с буферами хоста.

Параметр `UCX_RNDV_THRESH` задаёт порог переключения между eager- и rendezvous-протоколами: ниже порога используется eager, выше — rendezvous. В данной работе выбрано `UCX_RNDV_THRESH=0`, то есть rendezvous применяется для всех размеров сообщений. Это унифицирует маршрут передачи и убирает влияние точки переключения протоколов.

Параметр `UCX_RNDV_SCHEME=put_zcopy` задаёт схему передачи данных для rendezvous-протокола. В этой схеме отправитель иницирует запись данных в буфер получателя, а UCX по возможности выполняет передачу без дополнительного копирования через промежуточный буфер.

Параметр `UCX_MEMTYPE_CACHE=n` отключает кэширование типа памяти в UCX. Это заставляет библиотеку при каждой передаче заново определять, где расположен буфер (в памяти хоста или устройства), и снижает риск ошибочного выбора способа передачи для буферов устройств. Но это влечёт за собой небольшое увеличение накладных расходов.

После настройки UCX каждый процесс начинает инициализацию MPI-окружения.

5.3 Подготовка MPI-окружения

После `MPI_Init()` каждый процесс получает глобальный ранг и число процессов в `MPI_COMM_WORLD`.

Вызов `MPI_Comm_split_type(..., MPI_COMM_TYPE_SHARED, ...)` формирует локальный коммуникатор для каждого физического узла. С его помощью каждый процесс узнаёт свой локальный ранг среди процессов того же узла. Затем через `MPI_Allgather()` по локальному коммуникатору каждый процесс получает глобальные ранги своих соседей по узлу. На их основе формируется булев массив `on_my_node`, позволяющий при измерениях различать внутриузловые и межузловые пары GPU.

Планировщик Slurm [11] используется для выделения вычислительных ресурсов и задания соответствия между MPI-процессами и GPU. В batch-скрипте эти настройки задаются с помощью директив `#SBATCH`, а при интерактивном запуске могут передаваться как параметры команд `sbatch` или `srun`.

- `--gres=gpu:<число_GPU>` задаёт количество GPU, запрашиваемых на узле.
- `--gpus-per-task=1` указывает, что каждой MPI-задаче должен быть выделен ровно один GPU. При такой схеме Slurm ограничивает набор видимых устройств для процесса, в частности через переменную окружения `CUDA_VISIBLE_DEVICES`.
- `--gpu-bind=closest` задаёт привязку MPI-задач к ближайшим по топологии GPU относительно выделенных CPU-ядер.

В результате назначенный процессу GPU внутри его пространства видимости обычно получает локальный индекс 0, поэтому каждый процесс делает вызов `cudaSetDevice(0)`. После этого каждый процесс определяет имя своего хоста, число видимых GPU и PCI-идентификатор выбранного GPU. Эти сведения собираются всеми процессами с помощью `MPI_Allgather()` по `MPI_COMM_WORLD`. Поскольку PCI-идентификатор связан с положением устройства в аппаратной топологии узла, он позволяет при необходимости сопоставить результаты измерений с конкретным физическим устройством.

5.4 Реализация режима `one_to_one`

В режиме `one_to_one` в каждый момент времени измеряется передача только между одной упорядоченной парой устройств ($src \rightarrow dst$).

Процесс-*мастер* с рангом 0 организует обмены. Он последовательно перебирает все упорядоченные пары (src, dst), рассылает участникам задания и собирает результаты. При этом мастер остаётся полноценным участником измерений: если его ранг входит в текущую пару, он сам выполняет роль отправителя или получателя. Остальные процессы выступают *рабочими* — ожидают задание от мастера, выполняют назначенную роль и возвращают результат.

Перед началом измерений каждый ранг один раз выделяет два device-буфера: `d_send` для отправки и `d_recv` для приёма. Набор дополнительных буферов определяется средой передачи. В среде `auto` других буферов не требуется. В среде `host`, где данные обязаны проходить через оперативную память хоста, дополнительно используются хост-буферы, причём их вид зави-

сит от расположения пары:

- для межузловых пар — приватный хост-буфер `h_buf`;
- для внутриузловых пар — общий буфер `shared_h_buf`, размещённый в разделяемой памяти узла.

Разделение на два вида буферов не случайно. Общий буфер создаётся через `MPI_Win_allocate_shared()`: память для него выделяет локальный ранг 0, а остальные процессы того же узла получают указатель на эту область через `MPI_Win_shared_query()`. Благодаря этому отправитель и получатель, находящиеся на одном узле, обращаются к одной и той же области оперативной памяти, и получателю не нужно отдельно принимать данные по сети: он читает их прямо оттуда, куда отправитель уже скопировал их с GPU. Для межузловой же пары общая память узлам недоступна, поэтому используется приватный буфер с явной пересылкой по сети.

Измерение проводится отдельно для каждой пары (src, dst) , где $src \neq dst$. Мастер формирует задание и направляет его обоим участникам пары. Если одним из участников является сам мастер, задание отправляется только второму процессу. На протяжении всей серии активны только отправитель src и получатель dst . Лишь после завершения всех итераций мастер принимает результат и переходит к следующей паре — именно эта строго последовательная организация и гарантирует, что в каждый момент времени работает ровно одна пара.

В среде `auto` буферы `d_send` и `d_recv` передаются непосредственно в вызовы MPI: отправитель вызывает `MPI_Send()` для `d_send`, а получатель — `MPI_Recv()` для `d_recv`. Это справедливо как для внутриузловых, так и для межузловых пар.

В среде `host` для внутриузловой пары задействуется общий буфер. Отправитель копирует данные из `d_send` в `shared_h_buf` (копирование D2H), синхронизирует эту область вызовом `MPI_Win_sync()` и сообщает получателю о готовности данных коротким обменом `MPI_Sendrecv()`. Получатель дожидается этого сигнала, в свою очередь вызывает `MPI_Win_sync()` и копирует данные из общего буфера в `d_recv` (копирование H2D). Порядок вызовов здесь существен: сигнал готовности отправляется только после того, как запись в общий буфер завершена и синхронизирована, поэтому получа-

тель не начнёт чтение раньше, чем данные действительно окажутся в памяти.

В среде `host` для межузловой пары используется приватный буфер `h_buf`. Отправитель копирует данные из `d_send` в `h_buf` (D2H) и передаёт их через `MPI_Send()`. Получатель же принимает их в свой `h_buf` через `MPI_Recv()` и копирует в `d_recv` (H2D).

Время измеряемой итерации фиксируется на стороне получателя с помощью `MPI_Wtime()`: перед обменом сохраняется начальная отметка времени, после его завершения — конечная. Чтобы соседние итерации не накладывались друг на друга, после каждой итерации отправитель и получатель выполняют короткую синхронизацию через `MPI_Sendrecv()`.

По завершении серии получатель вычисляет статистические характеристики собранных значений задержки и передаёт результат мастеру. После обработки всех пар мастер рассылает рабочим процессам сигнал завершения, по которому все процессы освобождают выделенные ресурсы.

5.5 Реализация режима `all_to_all`

В режиме `all_to_all` все ранги участвуют в обмене одновременно: каждый ранг отправляет сообщение всем остальным рангам и принимает сообщения от всех остальных. В отличие от режима `one_to_one`, здесь одновременно активны все пары $(src \rightarrow dst)$ с $src \neq dst$, поэтому измеренная задержка отражает поведение коммуникационной среды уже в условиях конкуренции передач за общие ресурсы.

Перед началом измерений каждый ранг выделяет два `device`-буфера. Буфер `d_send` служит источником данных для всех отправок этого ранга, а `d_recv` — рабочим буфером назначения: в него копируется очередное принятое сообщение из соответствующего промежуточного буфера.

В среде `auto` для каждого из остальных $N - 1$ рангов выделяется отдельный `device`-буфер `recv_dev[src]`. Отдельный буфер на каждый источник необходим потому, что ранг принимает сообщения сразу от нескольких отправителей, а `MPI` требует для каждого незавершённого `MPI_Irecv()` собственный целевой буфер.

В среде `host` промежуточные буферы, как и в режиме `one_to_one`,

различаются по расположению пары. Для внутриузловых пар создаётся разделяемое MPI-окно через `MPI_Win_allocate_shared()`, однако, в отличие от `one_to_one`, здесь каждый локальный ранг выделяет в окне собственный участок памяти. Затем все процессы узла вызывают `MPI_Win_shared_query()` и получают указатели на участки остальных локальных рангов. Эти указатели сохраняются в массиве `shared_h_slots`, где элемент `shared_h_slots[src]` ссылается на общий буфер ранга `src`. Для межузловых пар используются приватные хост-буферы `send_host[peer]` и `recv_host[peer]`, создающиеся только для тех рангов `peer`, которые расположены на других узлах. Если всего запущено N процессов, а на узле текущего ранга находится L процессов, то для него выделяется $N - L$ буферов отправки и $N - L$ буферов приёма.

Каждая итерация обмена состоит из трёх последовательных фаз.

Сначала ранг заранее выставляет неблокирующие операции приёма для всех возможных источников — это позволяет не блокировать прогресс остальных передач. В среде `auto` для каждого источника вызывается `MPI_Irecv()` в буфер `recv_dev[src]`. В среде `host` для межузловых источников `MPI_Irecv()` выставляется в `recv_host[src]`, а для внутриузловых источников — нулевое сообщение через `MPI_Irecv()`, служащее лишь сигналом готовности данных в разделяемом буфере. Сами данные затем читаются напрямую из участка отправителя.

После регистрации всех приёмов ранг переходит к отправкам. Непосредственно перед этой фазой для каждого получателя фиксируется начальная отметка времени `MPI_Wtime()`. Далее в среде `auto` ранг выполняет `MPI_Isend()` из буфера `d_send`. В среде `host` для межузлового получателя данные сначала копируются из `d_send` в `send_host[dst]` (D2H), после чего `MPI_Isend()` выполняется для соответствующего хост-буфера. Для внутриузловых получателей ранг один раз за итерацию копирует данные из `d_send` в свой слот разделяемого окна, вызывает `MPI_Win_sync()` и отправляет каждому из них нулевое сообщение через `MPI_Isend()` — сигнал того, что данные в слоте готовы к чтению.

Завершившиеся приёмы обрабатываются в цикле с помощью `MPI_Waitany()`, который ожидает завершения любого из ранее выстав-

ленных `MPI_Irecv()` и возвращает индекс завершившегося запроса. Это позволяет обрабатывать сообщения в фактическом порядке их прихода. После завершения очередного приёма принятые данные копируются в `d_recv`: в среде `auto` — копированием D2D из `recv_dev[src]`; в среде `host` для межузловой пары — копированием H2D из `recv_host[src]`; для внутриузловой пары — после `MPI_Win_sync()` копированием H2D из слота отправителя. Сразу после копирования фиксируется конечная отметка времени.

Тем самым в режиме `all_to_all` начало измерения на ранге-получателе *dst* является общим для всех источников (ставится перед его фазой отправок), а конец фиксируется отдельно для каждой полученной передачи в момент обработки соответствующего приёма.

Завершив обработку всех приёмов в рамках текущей итерации, ранг дожидается завершения всех своих неблокирующих отправок через `MPI_Waitall()`.

По завершении серии каждый ранг вычисляет статистические характеристики для тех пар, где он выступал получателем, и передаёт их рангу 0. Ранг 0 собирает эти статистики. После этого все ранги освобождают выделенные ресурсы.

6 Методика эксперимента

6.1 Этапы

Серия измерений выполняется в два этапа. Сначала следует этап разогрева: измеряемый обмен выполняется `warmup` раз без фиксации времени. Эти итерации позволяют исключить из статистики возможные начальные накладные расходы. Затем следует измерительный этап: обмен повторяется `iters` раз, и задержка каждой итерации сохраняется для последующего вычисления статистик [4].

6.2 Выбор количества итераций

Количество итераций выбирается как компромисс между точностью оценок и длительностью эксперимента. При слишком малом числе измерительных итераций разброс выборки обычно велик, а воспроизводимость результатов между запусками снижается. При слишком большом числе итераций общее время тестирования заметно растёт, тогда как выигрыш в точности постепенно насыщается.

Альтернативой служат различные критерии останова, завершающие серию при стабилизации оценок. В работе [3] показано, что задержки в коммуникационной среде кластера могут обладать выраженной цикличностью, и предлагается спектральный анализ для обоснования критерия останова.

В настоящей работе из соображений простоты используется фиксированное число итераций.

6.3 Обработка результатов

Для оценки типичного времени передачи предпочтительнее использовать медиану, а не среднее [4]. В серии измерений основная часть значений обычно сосредоточена в одной области, однако отдельные итерации могут давать заметно большие задержки. Под влиянием редких больших значений среднее может смещаться, тогда как медиана остаётся более устойчивой к ним. Для оценки разброса рассматривается стандартное отклонение.

Для формального описания вычисляемых статистик приведём используемые формулы. Пусть $t_1, \dots, t_n$ обозначает временной ряд задержек. Через

$$t_{(1)} \leq t_{(2)} \leq \dots \leq t_{(n)}$$

обозначим соответствующий вариационный ряд.

Тогда медиана определяется как

$$med = \begin{cases} t_{(\frac{n+1}{2})}, & n \text{ нечётно,} \\ \frac{t_{(\frac{n}{2})} + t_{(\frac{n}{2}+1)}}{2}, & n \text{ чётно.} \end{cases}$$

Для оценки разброса значений вычисляется выборочное стандартное отклонение

$$std = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (t_i - \bar{t})^2}, \quad \bar{t} = \frac{1}{n} \sum_{i=1}^n t_i.$$

6.4 Ограничения

Не все факторы можно контролировать в рамках эксперимента:

- точный маршрут передачи зависит от топологии кластера, настроек маршрутизации и текущего состояния сети. Без административного доступа его трудно отследить, в связи с чем диагностика причин повышенных задержек в коммуникационной среде усложняется;
- разделить задержку передачи и накладные расходы затруднительно: фоновая активность других задач на узле и процессов в сети вносит шум в задержки.

7 Среда проведения эксперимента

7.1 Аппаратная среда

Эксперименты проводились на многоузловом GPU-кластере. Каждый вычислительный узел содержит два CPU, восемь GPU и шесть NVSwitch. Внутри узла каждое GPU соединено с каждым NVSwitch, поэтому их внутриузловую топологию можно представить как полный двудольный граф $K_{8,6}$.

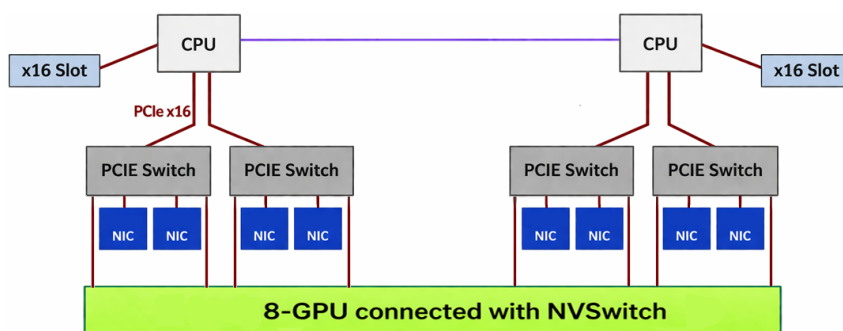


Рис. 2: Пример внутриузловой топологии GPU-узла (Источник: [13])

Между узлами обмен выполняется через сетевые адаптеры и коммутируемую сеть. Используется двухуровневая топология Клоза (leaf-spine). В ней вычислительные узлы подключаются к коммутаторам первого уровня, а коммутаторы первого уровня связаны с коммутаторами второго уровня.

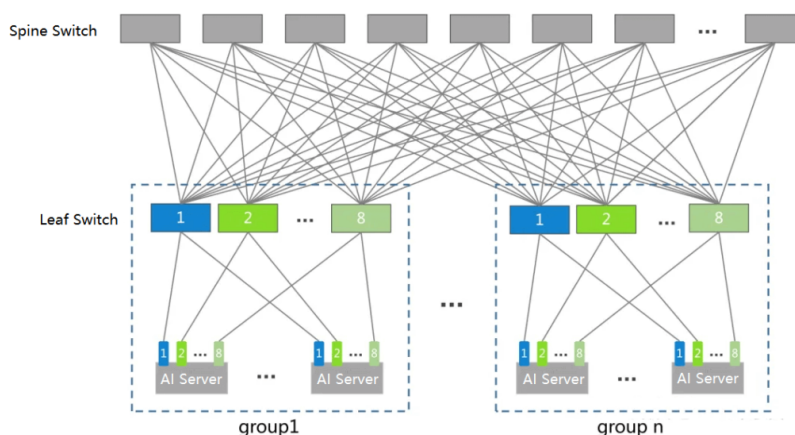


Рис. 3: Схематическое изображение двухуровневой топологии Клоза (по материалам [14]).

Точные модели оборудования не указываются в работе из соображений конфиденциальности.

7.2 Программная среда

Ниже приведены версии программного обеспечения, использованных при сборке и запуске модуля, а также ключевые параметры сборки UCX и Open MPI.

Компонент	Версия/настройка
Open MPI	5.0.10
UCX	1.15.0
CUDA Toolkit	12.2

Таблица 2: Программная среда проведения эксперимента.

Параметры сборки UCX:

- `--with-cuda`: поддержка CUDA и работы с GPU-памятью;
- `--with-verbs`: поддержка InfiniBand.

Параметры сборки Open MPI:

- `--with-ucx`: использование UCX как транспортного слоя;
- `--with-cuda`: поддержка CUDA-aware MPI для передачи GPU-буферов;
- `--with-slurm`: интеграция со Slurm для запуска через `srun`.

8 Результаты эксперимента

Пары устройств всюду записываются в формате $N.K$, где N — номер вычислительного узла, а K — локальный индекс GPU на этом узле.

8.1 Зависимость от размера сообщения

В этом разделе рассматривается зависимость медианы задержки от размера сообщения в режиме `one_to_one` для двух пар: внутриузловой $20.0 \rightarrow 20.1$ и межузловой $20.0 \rightarrow 21.0$.



Рис. 4: Медиана задержки от размера сообщения. Пара $20.0 \rightarrow 20.1$ (внутриузловая), среда `host`, режим `one_to_one`.

На рис. 4 показана типичная форма зависимости: при малых размерах задержка практически не зависит от объёма — преобладают постоянные накладные расходы на инициацию обмена.

Таблица 3 содержит медианы задержек для четырёх конфигураций (два типа пар $\times$ две среды) при определённых размерах сообщений.

Пара	Среда	1 КБ	8 КБ	64 КБ	512 КБ	≈ 4 МБ
внутриузловая (20.0→20.1)	auto	12.1	12.2	12.3	12.7	15.4
	host	22.5	23.8	24.9	59.0	331.7
межузловая (20.0→21.0)	auto	14.4	15.2	16.7	32.8	161.1
	host	31.2	33.1	36.1	87.0	484.1

Таблица 3: Медиана задержки (мкс), режим one_to_one, UCX_RNDV_THRESH=0.

Для внутриузловой пары в среде auto задержка при малых размерах составляет около 12 мкс и почти не изменяется. В среде host задержки заметно выше и растут значительно быстрее из-за обязательных копирований D2H и H2D.

Для межузловой пары обе среды демонстрируют более высокие задержки и более резкий рост. Так, при ≈ 4 МБ auto и host расходятся примерно втрое.

Отметим, что при варьировании порога UCX_RNDV_THRESH в среде auto локальный пик смещается вслед за выбранным пороговым значением. Это соответствует переходной зоне eager→rendezvous в UCX.

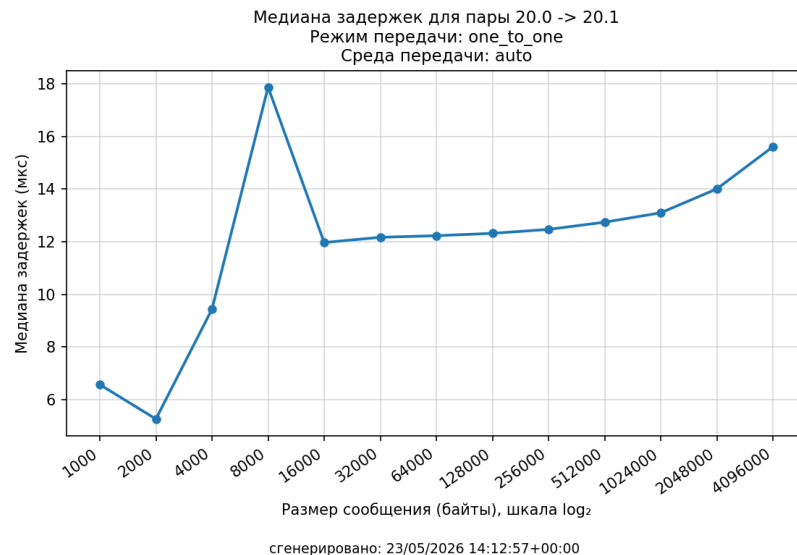


Рис. 5: Пара 20.0 → 20.1 (внутриузловая), среда auto, порог UCX_RNDV_THRESH по умолчанию. Виден локальный пик в районе 8 КБ.

8.2 Зависимость от режима тестирования

На теплокартах 6 и 7 приведены медианы задержек для фиксированного размера сообщения 16 МБ. Белая диагональ соответствует передаче устройства самому себе, и в работе такой тип передачи не рассматривался. По оси Y отложены GPU-источники. По оси X — GPU-получатели. Таким образом, матрица разбивается на четыре блока: $20.* \rightarrow 20.*$, $20.* \rightarrow 21.*$, $21.* \rightarrow 20.*$ и $21.* \rightarrow 21.*$.

Режим one_to_one

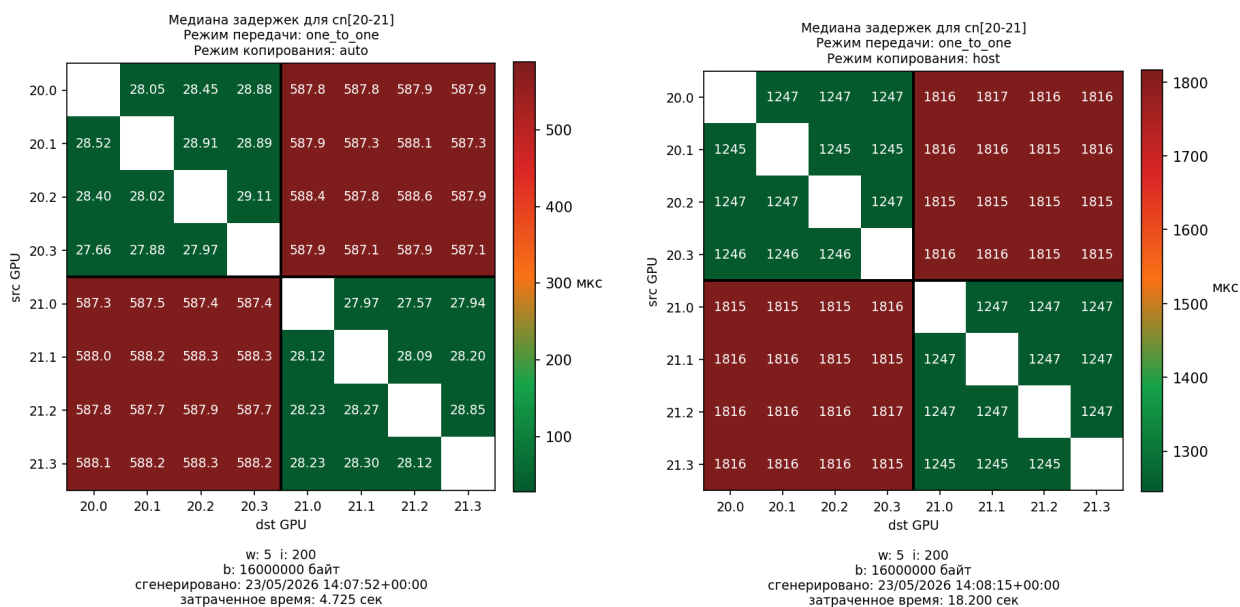


Рис. 6: Теплокарты медианы задержек для режима one_to_one, 16 МБ: слева — среда auto, справа — среда host.

В среде auto (левая теплокарта на рис. 6) внутриузловые передачи имеют наименьшую задержку: медиана составляет около 28 мкс. Межузловые передачи в той же среде заметно медленнее — около 588 мкс.

В среде host (правая теплокарта на рис. 6) картина иная: внутриузловые передачи имеют задержку около 1.25 мс, межузловые — около 1.8 мс. Разница между внутриузловыми и межузловыми передачами оказывается значительно меньше, чем в auto.

Режим all_to_all

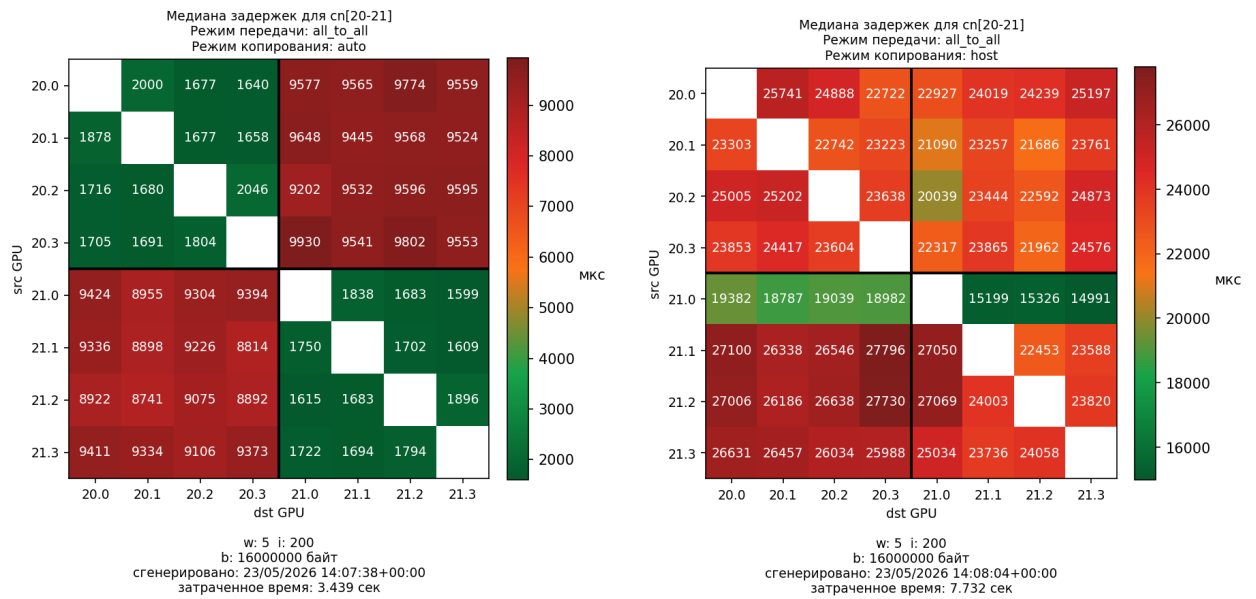


Рис. 7: Теплокарты медианы задержек для режима all_to_all, 16 МБ: слева — среда auto, справа — среда host.

При переходе к all_to_all значения медианы задержки заметно возрастают во всех средах. Более того, в среде auto контраст между внутриузловым и межузловым блоками выражен сильнее, чем в host.

Заключение

В настоящей курсовой работе была рассмотрена задача измерения и анализа задержек передачи данных для внутриузловых и межузловых пар GPU.

Основные результаты работы можно сформулировать следующим образом.

1. Вычислительная система формализована как граф $G = (V, E)$, на котором введены понятия маршрута и среды передачи, а также определена задержка.
2. Проведён обзор некоторых инструментов исследования коммуникационной среды вычислительных кластеров. Проанализированы существующие подходы к измерению задержек.

3. Существующий программный модуль доработан. По сравнению с подходом [1] в работе внесены следующие изменения:
 - среда передачи выделена как самостоятельный параметр эксперимента (auto/host);
 - схема «один MPI-процесс на узел» заменена схемой «один MPI-процесс на один GPU», что потребовало переработки режимов one_to_one и all_to_all.
4. Проведена серия экспериментов при различных размерах сообщений, средах передачи и режимах тестирования. Полученные данные представлены в виде графиков зависимости медианы задержки от размера сообщения и теплокарт, показывающих распределение задержек по парам GPU.
5. Реализован экспорт результатов в формат NetCDF, что позволяет хранить результаты измерений в структурированном виде и использовать их для дальнейшего анализа и визуализации.

В дальнейшем работу можно развивать в нескольких направлениях.

1. Реализовать критерий останова серий измерений по аналогии с подходом из [3].
2. Расширить набор режимов тестирования. Например, режим one_to_all (аналог broadcast) позволит измерить задержку при одновременной рассылке данных от одного GPU всем остальным, а режим all_to_one (аналог gather) — при одновременном сборе данных от всех GPU к одному приёмнику.

Литература

Список литературы

- [1] Бегаев А. А., Сальников А. Н. Метод измерения задержек при передаче данных между графическими процессорами, находящимися на разных узлах вычислительного кластера [Электронный ресурс]. — URL: <http://cyberleninka.ru/article/n/metod-izmereniya-zaderzhek-pri-peredache-dannyh-mezhdu-graficheskimi-protssessorami-nahodyaschimisya-na-raznyh-uzlah-vychislitelnogo/viewer> (дата обращения: 24.05.2026).
- [2] Сальников А. Н., Андреев Д. Ю., Лебедев Р. Д. Инструментальная система для анализа характеристик коммуникационной среды вычислительного кластера на основе функций стандарта MPI [Электронный ресурс]. — URL: <https://cyberleninka.ru/article/n/instrumentalnaya-sistema-dlya-analiza-harakteristik-kommunikatsionnoy-sredy-vychislitelnogo-klastera-na-osnove-funktsiy-standarta-mpi> (дата обращения: 24.05.2026).
- [3] Волчанинов А. П., Сальников А. Н. Исследование структуры серии измерений задержки при нагрузочном тестировании коммутационной среды вычислительного кластера // Труды RuSCDays-2023. — 2023. — С. 42 [Электронный ресурс]. — URL: https://2023.russianscdays.org/files/2023/RuSCDays23_Proceedings.pdf (дата обращения: 24.05.2026).
- [4] Hoefler T., Belli R. Scientific Benchmarking of Parallel Computing Systems // SC '15: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. — 2015 [Электронный ресурс]. — DOI: <https://doi.org/10.1145/2807591.2807644> (дата обращения: 24.05.2026).
- [5] Sojoodi A., Akbari M., Sharifian H., Farazdaghi A., Grant R. E., Afsahi A. Accelerating Intra-Node GPU Communication: A Performance Model for Multi-Path Transfers // SC Workshops '25. — 2025. — P. 449–460 [Элек-

тронный ресурс]. — DOI: <https://doi.org/10.1145/3731599.3767392> (дата обращения: 24.05.2026).

- [6] Чулкевич Р. А., Козырев В. И., Костенецкий П. С., Раимова А. А. Экспериментальная оценка результатов внедрения технологии NVIDIA GPUDirect на суперкомпьютере НИУ ВШЭ // Суперкомпьютерные дни в России: Труды международной конференции. — Москва: МАКС Пресс, 2023. — С. 186–194 [Электронный ресурс]. — URL: <https://publications.hse.ru/pubs/share/direct/890513978.pdf> (дата обращения: 24.05.2026).
- [7] Open MPI: CUDA-Aware Support [Электронный ресурс]. — URL: <https://www.open-mpi.org/faq/?category=runcuda> (дата обращения: 24.05.2026).
- [8] Unified Communication X [Электронный ресурс]. — URL: <https://openux.org/> (дата обращения: 24.05.2026).
- [9] NVIDIA. CUDA Runtime API Reference [Электронный ресурс]. — URL: <https://docs.nvidia.com/cuda/cuda-runtime-api/> (дата обращения: 24.05.2026).
- [10] NVIDIA. GPUDirect RDMA [Электронный ресурс]. — URL: <https://docs.nvidia.com/cuda/gpudirect-rdma/> (дата обращения: 24.05.2026).
- [11] Slurm Workload Manager. Quick Start User Guide [Электронный ресурс]. — URL: <https://slurm.schedmd.com/quickstart.html> (дата обращения: 24.05.2026).
- [12] NVIDIA Developer Blog. HGX-2 Fuses HPC and AI Computing Architectures [Электронный ресурс]. — URL: <https://developer.nvidia.com/blog/hgx-2-fuses-ai-computing> (дата обращения: 24.05.2026).
- [13] NVIDIA Developer Blog. Introducing NVIDIA HGX A100: The Most Powerful Accelerated Server Platform for AI and High Performance Computing [Электронный ресурс]. — URL: <https://developer.nv>

- idia.com/blog/introducing-hgx-a100-most-powerful-accelerated-server-platform-for-ai-hpc/ (дата обращения: 24.05.2026).
- [14] Fibermall Blog. Fibermall Delivers HPC Networking Solutions for AIGC [Электронный ресурс]. — URL: <https://www.fibermall.com/blog/fibermall-delivers-hpc-networking-solutions-for-aigc.htm> (дата обращения: 24.05.2026).
- [15] Unidata. NetCDF Documentation [Электронный ресурс]. — URL: <https://docs.unidata.ucar.edu/netcdf-c/current/> (дата обращения: 24.05.2026).
- [16] p2pBandwidthLatencyTest [Электронный ресурс]. — URL: https://github.com/NVIDIA/cuda-samples/blob/master/cpp/5_Domain_Specific/p2pBandwidthLatencyTest/p2pBandwidthLatencyTest.cu (дата обращения: 24.05.2026).
- [17] nvbandwidth [Электронный ресурс]. — URL: <https://github.com/NVIDIA/nvbandwidth> (дата обращения: 24.05.2026).
- [18] OSU Micro-Benchmarks [Электронный ресурс]. — URL: <https://mvapich.cse.ohio-state.edu/benchmarks/> (дата обращения: 24.05.2026).
- [19] Clustbench / network-tests2 [Электронный ресурс]. — URL: <https://github.com/clustbench/network-tests2> (дата обращения: 24.05.2026).

Приложение А. Запуск программного модуля

А.1 Компиляция проекта

Сборка GPU-модуля выполняется в каталоге `gpu_tests`. Бинарный файл `gpu` собирается скриптом `build.sh`, который компилирует исходные файлы `gpu_benchmark.cpp`, `gpu_common.cpp`, `gpu_one_to_one.cpp`, `gpu_all_to_all.cpp` через `mpicxx`.

А.2 Постановка задачи в Slurm

Для прогона интересующих комбинаций параметров используется скрипт `task.sbatch`. Ключевая логика запуска задаётся следующими блоками:

- массивы `ENVS`, `MODES`, `CASES`, определяющие перебор конфигураций;
- запуск `gpu` через `srun` с параметрами `--bytes`, `--warmup`, `--iters`, `--env`, `--mode`, `--out`.

А.3 Визуализация результатов

Для построения теплокарт используется скрипт `gpu_heatmap.py`.

Для построения графиков зависимости задержки от размера сообщения по выбранной паре GPU используется `gpu_plot.py`.

Для экспорта матриц задержек и сопутствующих метаданных в формат NetCDF используется модуль `gpu_netcdf.py`.

А.4 Просмотр .nc-файлов

Для просмотра файлов в формате NetCDF можно использовать утилиту `ncview` с графическим интерфейсом. Она позволяет открыть файл `.nc` и просматривать значения переменных в виде двумерных срезов. В контексте данной работы это удобно для просмотра теплокарт задержек по парам GPU.